

Corroborate: A Framework for Testing Mechanism Claims in Reinforcement Learning

Jean Seong Bjorn Choe¹

rnilva849@gmail.com

¹Independent Researcher

Abstract

Reinforcement learning research often studies the mechanism behind a proposed method, not only its benchmark score. But the theorem behind a mechanism is proven under conditions the implementation relaxes, so whether the analytical prediction holds in the practical regime is an open empirical question. It is also a difficult one. The relevant quantities are entangled and confounded, and testing the link between the mechanism and the outcome is closer to causal inference than to plotting curves. Because the test is non-standard, the apparatus built for it in one paper is rarely reusable by another, and the libraries the field runs on are built to reproduce benchmark outcomes, not to test mechanisms. We present `corroborate`¹, a framework for the part these libraries do not serve. It combines three principles, each individually recognised in the methodology literature. First, it isolates a single mechanism as a functional unit that can be swapped without disturbing the rest of the algorithm, so the comparison is a controlled causal contrast. Second, it codifies a hypothesis as a small program whose assumptions are visible and which runs as a test. Third, it treats "we cannot tell" as a distinct verdict that carries through a chain of claims, instead of letting it collapse into "no effect." We demonstrate the framework on Double-DQN, where it renders the known verdict as explicit per-environment claims and reports that the test of the analytical prediction, that bias reduction improves outcome, is underpowered in most environments. In one long-horizon regime where the algorithm is harmful, the framework separates a working mechanism from a worse outcome. Bias is reduced, yet an experiment that continuously dials the mechanism's strength shows that the reduction trades against return, and a single-edge intervention localises the harm to the evaluator.

1 Introduction

Establishing that an algorithm's benefit stems from its proposed mechanism is not trivial. The difficulty has structural sources. (1) The theorem behind a mechanism is proven under conditions that can rarely be held in practice (Baird, 1995; Sutton & Barto, 2018), so its conclusion is downgraded from a proven result to an analytical prediction to be tested. (2) The mechanism is entangled with the rest of the implementation, so the swap meant to isolate it changes other things at the same time. Studies that control for these details find that what drives performance is often not the headline mechanism but the surrounding optimisations (Engstrom et al., 2020; Andrychowicz et al., 2021; Huang et al., 2022a). (3) The effect, where it exists, is often small relative to the noise across seeds and environments, so the comparison is underpowered and point estimates are misleading (Agarwal et al., 2021). Taken together, these difficulties turn the question of whether the analytical prediction holds in practice into a causal-inference problem.

¹Code and supplementary material at <https://github.com/rnilva/corroborate>

The field’s methodologists have recognised this and called for experiments that explain an algorithm’s behaviour, not only rank it (Patterson et al., 2024; Jordan et al., 2024). Jordan et al. (2024) further show that rigorous benchmarking is often computationally prohibitive and demonstrate that scientific testing yields insights that benchmarking cannot. Yet the apparatus for such testing is built from scratch in each paper, in a form that does not transfer, and the field’s standard libraries are built to reproduce benchmark outcomes rather than to test mechanisms. Reproducing an outcome is a different question from testing a mechanism. That PPO’s reported scores replicate does not settle whether its clipping mechanism does what the community reads into it, and exposing that gap took dedicated follow-up work (Engstrom et al., 2020). What is missing is infrastructure in which a mechanism claim can be written, run, and composed.

We present `corroborate`, a framework built on three principles, each answering one of the difficulties above. First, it makes a single mechanism the unit of intervention, so a swap is a clean causal contrast rather than a blurred algorithm swap. Second, it codifies a hypothesis as a small program whose scope and predicted direction are visible and run as a test rather than described in a methods section. Third, it treats an inconclusive result as a distinct verdict that propagates through a chain rather than collapsing into "no effect." None of the three is novel alone. The contribution is the framework in which they compose.

We demonstrate the framework on Double-DQN, whose overestimation-bias mechanism (van Hasselt, 2010; van Hasselt et al., 2016) is well understood. A method’s verdicts can be trusted for an open question only after they have been checked on a settled one. Most of what the framework reports there is the field’s existing understanding, rendered as explicit per-environment claims rather than a scalar. DDQN reduces overestimation bias in most environments. Its benefit to outcome is weak and environment-conditional, with no effect at the pooled level. The test of the analytical prediction that bias reduction improves outcome is underpowered in most environments, and the framework reports it as such.

One regime is the exception. At a long horizon ($\gamma = 0.999$), Double-DQN becomes harmful in one environment, and there the framework produces a powered, directional result that a scalar verdict would obscure. The mechanism still operates, in that DDQN measurably reduces overestimation, yet the outcome is worse. A controlled experiment on the greedification component alone shows that strengthening the mechanism trades bias for harm monotonically, and a single-edge intervention localises the harm to the evaluator. We do not establish why, and with one harmful environment we claim no generalisation. This paper establishes that such testing can be written and run. Applying it to mechanisms that are not yet understood is the next step.

Concretely, we make four contributions. First, a decomposition discipline that makes a single mechanism the unit of intervention, with component boundaries drawn at theorems (§3.1). Second, executable claims, called bridges, whose commitments of scope, evidential tier, and predicted direction are declared before any data is seen, and whose mapping from statistic to verdict is inspectable code (§3.2). Third, a three-valued verdict semantics comprising `HELD`, `NO_EFFECT`, and `POWER_INSUFFICIENT` that propagates inconclusive evidence through composed claims (§3.3). Fourth, a case study on Double-DQN that calibrates the framework against the settled $\gamma = 0.99$ picture and, at $\gamma = 0.999$, isolates a powered dissociation between a working mechanism and a harmful outcome (§4).

2 Related Work

Methodology and rigour. A substantial line of work has improved how RL results are measured. It includes controlling seed variance and reporting interval estimates rather than point estimates (Agarwal et al., 2021), disciplined hyperparameter and experiment design (Patterson et al., 2024), and the argument that benchmarking alone is an insufficient basis for claims (Jordan et al., 2024). That line of work concerns the reliability of the outcome metric, and `corroborate` is orthogonal to it. The framework does not improve how an outcome is measured but supplies a form in which the claim that a mechanism produces the outcome can be stated and tested. The call by Jordan et al.

(2024) for scientific testing as a complement to benchmarking is the closest in spirit, and we provide infrastructure for it.

Causal inference for evaluation. In RL, causal methods mostly appear under the heading of causal RL, using causal structure to improve an agent (e.g., counterfactual reasoning, causal world models). Our use is different. We bring standard causal inference tools (partial correlation, conditional independence structure discovery, mediation) to evaluating an algorithm rather than to its learning. The interventional contrast we rely on rests on the single-component swap of Section 3.1. The mediator-to-outcome edge remains an identification question addressed separately for each claim, and the swap does not settle it. Our per-window mediation analysis, which tracks how a mediating relationship changes over training, follows time-varying mediation analysis (Lindquist, 2012; VanderWeele & Tchetgen Tchetgen, 2017). We do not contribute new causal estimators. We bring existing ones to bear in a form that composes.

Infrastructure. The field’s widely used libraries serve training and benchmark reproduction (for example, CleanRL (Huang et al., 2022b), Stable-Baselines3 (Raffin et al., 2021), and Acme (Hoffman et al., 2020)), and experiment trackers serve logging. By contrast, `corroborate` runs the same kind of training machinery as an active intervention rather than as a benchmark to reproduce (Section 3.1). Because the framework owns its runs, the run set it accumulates is interventional, and comparisons rest on interventional contrasts rather than on observational data. Where these libraries surface mechanism-level quantities at all, such as value estimates or temporal-difference errors, they do so as diagnostics off a benchmark run, not as claims tested against an interventional one. The framework fills that gap.

3 The framework

The three subsections take `corroborate`’s principles in turn. Section 3.1 isolates a mechanism as a unit of intervention, Section 3.2 codifies a hypothesis as an executable claim, and Section 3.3 propagates inconclusive verdicts through composed claims. Three terms recur. A *condition* is one algorithm variant under study, such as vanilla DQN or DDQN, and each environment-condition pair is populated by a set of seeded runs. An evaluation *window* is a fixed block of greedy evaluation episodes run at a regular interval during training, and per-window metrics are computed within one such block (the released code names these windows *bursts*, as in `eval_best_burst_raw_mean`). The *run set* is the logged collection of all runs, against which claims are tested.

3.1 Isolating a mechanism as a unit of intervention

A reinforcement learning implementation interleaves components of different theoretical status. The paper-claimed mechanism sits alongside code-level optimisations for training speed, scaffolding for software-engineering reasons, and helpers for analysis. Conventional implementations, whether layered frameworks or single-file scripts, mix these freely, which both blurs what the theory claims against what the engineering arranges and makes intervention studies hard to credit. When Engstrom et al. (2020) found that PPO’s clipping mechanism could not be cleanly credited for its gains, the reason was of this kind. The algorithm swap carried a dozen code-level optimisations along with it, so the mechanism was confounded with components of a different origin.

To make the clean intervention the natural one, `corroborate` decomposes the algorithm so that component boundaries fall at the theorems, separating mechanism from mechanics. Each slot is a `@claim`, the operation a theorem makes a claim about, behind a typed `Protocol` contract. Operations that carry no theorem, such as parameter allocation or buffer bookkeeping, are kept out of the claim slots as mechanics. The Bellman target, for instance, is split into a `greedification` slot and a `gradient_rule` slot because the overestimation-bias result (van Hasselt, 2010) and the semi-gradient choice (Sutton & Barto, 2018) are two distinct theoretical commitments. The

Double-DQN mechanism is then one slot’s content. DDQN versus vanilla is one theorem’s worth of difference, the decoupling of action selection from value evaluation ([van Hasselt et al., 2016](#)), and that difference lives in the `greedification` slot.

```
Intervention(  
  slot_path='bootstrap',  
  replacement=partial(bootstrap, greedification=double_greedify),  
)
```

Because `max_greedify` and `double_greedify` satisfy the same protocol with identical signatures, the swap is a single keyword, visible at the call site. This does not, and could not, guarantee that the intervention is truly atomic. Whether a swap of one slot changes only one thing depends on how faithfully the algorithm was authored into the slots, and the type system checks the declared boundary, not its behaviour. What the decomposition does is make the single-component swap the default and make a swap that reaches into other components visible rather than silent, so a researcher using the framework as intended obtains an interventional contrast without having to engineer and police that cleanliness by hand. Two further limits are worth stating. First, the cleanliness is internal to the framework. The compared conditions differ in one slot because they are the same re-implementation with one component changed, not a re-run of the original codebases, and fidelity to [Mnih et al. \(2015\)](#) and [van Hasselt et al. \(2016\)](#) is a separate question documented in the slot docstrings (Appendix B gives the full decomposition). Second, the decomposition assumes that component boundaries can be drawn at theorems at all. Section 5 discusses where that assumption strains.

3.2 Codifying a hypothesis as an executable claim

A mechanism paper provides each element of its claim, namely a theorem, its proof, and empirical curves. But the elements are rarely organised into a single object that can be tested as a chain. They sit in separate artefacts. The bound lives in a lemma, and the bias and the score sit in different figures, with no connective structure joining premise to mechanism to outcome. A paper need not formalise its chain. But to test a mechanism rather than restate it, the chain has to exist as an object, and its absence makes the test hard to get right. The framework codifies each edge of the chain as a small program called a *bridge*, whose decorator declares the claim’s commitments and whose body is the test. One such edge is the *link* edge, which states that under the vanilla baseline, across environments, lower bias tends to accompany higher outcome. This edge establishes that bias is outcome-relevant at all, the premise that reducing it could help. It is a within-baseline association, not a causal claim, and the DDQN intervention is tested at separate edges. Figure 1 shows how it is authored, lightly simplified.

The claim is testable in two senses. The first is execution, in which the bridge runs against the run set and returns a verdict. The second is inspection. The decorator’s commitments are legible before any data is seen. The declaration `tier=ASSOCIATIONAL` marks this edge as a within-baseline correlation rather than an interventional contrast, so a reader knows not to read causation into it, and `predicted_direction='a_lt_b'` fixes the sign in advance, so the verdict cannot be chosen after the fact. The body makes the claim something a reviewer can read and contest, because the mapping from statistic to verdict is explicit. One can object that the threshold is wrong, that the significance bar is too lax, or that a sign-flip should not be grouped with a null. Each of these objections has a concrete target, because the mapping is a dozen lines of code rather than a methods paragraph.

The scope argument specifies which environments and conditions a claim applies to. This link edge concerns the baseline, so it is scoped to the vanilla condition (`VANILLA_ONLY`). A claim about where a theorem’s premise holds would instead declare the corresponding scope. Declaring scope on the bridge, before any verdict, makes such conditioning a visible commitment that a reader can contest, rather than a filter quietly applied after seeing the data. Appendix A gives the full composed chain.

```

@claim_bridge(
    source='jensen_gap', target='eval_best_burst_raw_mean',
    direction=Direction.INVERSE, tier=Tier.ASSOCIATIONAL,
    scope=VANILLA_ONLY,
    predicted_direction='a_lt_b',
)
def bias_predicts_worse_outcome__vanilla(ps: PartialSpearmanResult):
    rho, p = ps.rho_pooled, ps.p_value
    if isnan(rho) or isnan(p): return POWER_INSUFFICIENT
    if rho <= -0.3 and p <= 0.05: return HELD
    if rho >= 0.3: return NO_EFFECT, SIGN_FLIP
    if abs(rho) <= 0.1: return NO_EFFECT, NULL_EFFECT
    return POWER_INSUFFICIENT

```

Figure 1: The link edge as an executable claim. The decorator declares the claim’s commitments (tier, scope, predicted direction) before any data is seen. The body makes the mapping from statistic to verdict explicit and contestable.

Claim	Verdict	Evidence
Mechanism reduces bias (direction)	HELD	10/12 envs, sign test $p = 0.019$
Mechanism reduction (magnitude)	NO_EFFECT	$I^2 = 0.97$, prediction interval crosses 0
Outcome: DDQN > vanilla	env-conditional	$P(D > V) = 0.59$, perm. $p = 0.003$
Outcome benefit (magnitude)	NO_EFFECT	both metrics, under heterogeneity

Table 1: Calibration on the settled $\gamma = 0.99$ suite. The consensus picture is expressed as per-environment verdicts rather than a scalar. The mechanism’s *direction* holds broadly while its *magnitude* is not poolable, and the outcome benefit is environment-conditional. The framework reports “no poolable population effect” (NO_EFFECT on magnitude, under $I^2 = 0.97$ heterogeneity) rather than reading a non-significant pool as zero.

3.3 Propagating inconclusive verdicts through composed claims

A test can fail to find an effect for two different reasons. Either the effect is absent, or the data cannot tell. Standard practice collapses these, reporting “no significant difference” as “no difference,” and does so silently. Along a chain the collapse compounds, so that a link that was merely underpowered becomes, downstream, a confident claim that the chain does not hold. The framework keeps the two apart. Every bridge returns one of three verdicts: HELD, NO_EFFECT, or POWER_INSUFFICIENT. A NO_EFFECT is tagged with why, since a null and a sign-flip are different refutations. POWER_INSUFFICIENT is a distinct verdict, not a softened null.

Bridges compose into a chain, and the chain’s verdict is built from its edges. The chain holds only if every edge holds, and an underpowered edge renders the whole chain inconclusive rather than refuted. The framework can therefore report “we cannot tell whether this chain holds” rather than silently “it does not,” carrying the discipline of equivalence testing (Lakens, 2017) from a single comparison through to the composed claim. One distinction this makes precise matters for the case study. A chain that runs premise to mechanism to outcome tests an indirect effect through the mediator, which is a different claim from the algorithm’s total effect on outcome (Pearl, 2009). A chain can hold as a mediated path while the total effect does not, and Section 4 turns on exactly that gap.

Environment	Outcome d	Residual bias	Absolute bias
Asterix	-0.80	40.8	231
MetaMaze	-0.08	8.2	3
Freeway	0.10	12.5	3
LunarLander	0.25	3.6	241
PacMan	0.38	1.1	153
Breakout	0.66	7.7	44
FourRooms	1.35	4.9	1
SpaceInvaders	2.16	3.6	54

Table 2: Overestimation alone does not account for the $\gamma = 0.999$ harm. The rank correlation between outcome and residual bias is $\rho = -0.62$ ($p = 0.10$), and against absolute bias $\rho = -0.21$ ($p = 0.61$). At $n = 8$ both fall through the thresholds of Figure 1 to POWER_INSUFFICIENT, and we read neither as a null. The environment-level pattern carries the point: LunarLander’s absolute bias exceeds Asterix’s and PacMan’s is comparable (241 and 153 against 231), yet the harm appears only at Asterix. Residual bias is the relative overestimation $(Q - MC)/|MC|$ left under DDQN. Absolute bias is the vanilla magnitude.

4 Case Study: Double-DQN

We choose Double-DQN as a case study because its mechanism is well defined and its implementation straightforward. The claim under test is the Double-DQN chain (van Hasselt et al., 2016). Its premise is maximisation bias, in which the max operator overestimates in proportion to estimation noise (van Hasselt, 2010). Its mechanism decouples action selection from value evaluation to reduce that overestimation. Its outcome is the improved return that the reduction is meant to produce. Throughout, bias is measured as the *Jensen gap*, the gap between the learned value estimate and the realised Monte-Carlo return. Outcome is summarised per run in two ways: the mean return of the best evaluation window, which follows the reporting protocol of van Hasselt et al. (2016), and the mean over the last 30% of windows, a stability-oriented late-training aggregate. These are the two metrics of Table 1.

4.1 Calibration at $\gamma = 0.99$

On the canonical $\gamma = 0.99$ suite, the framework reproduces the field’s understanding as explicit per-environment verdicts (Table 1, with the full suite in Appendix C). DDQN’s bias reduction holds in direction but is not poolable in magnitude, and its outcome benefit is environment-conditional, with both metrics pooling to NO_EFFECT under heterogeneity. This is the consensus verdict, modest and mixed, stated as per-environment claims rather than a scalar. The calibration warrants the rest of the study. Where the answer is known the verdicts behave as they should, including reporting no poolable population effect rather than reading a non-significant pool as zero.

4.2 A powered dissociation at $\gamma = 0.999$

At the long horizon the pattern changes. At $\gamma = 0.999$, Asterix, which DDQN helps at $\gamma = 0.99$, becomes the sole adequately powered environment where DDQN is harmful ($d = -0.80$, $n = 30$). The other three environments that reach adequate power for this contrast, Breakout, FourRooms, and SpaceInvaders, still show a benefit, and the remaining rows of Table 2 are underpowered at this horizon, small in magnitude and mixed in sign. The mechanism, however, still operates everywhere, including at Asterix, where DDQN reduces relative overestimation by 46% (to the residual 40.8 of Table 2). The mechanism works while the outcome worsens, with adequate power on both measurements. This is a dissociation that a scalar benefit number would average away.

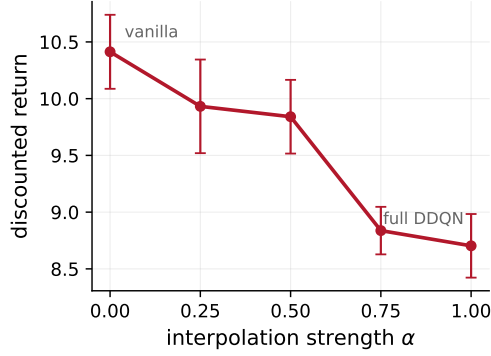


Figure 2: Graded-strength intervention on Asterix ($\gamma = 0.999$). Best-window discounted return falls monotonically as the greedification interpolation strength α rises from vanilla (0) to full DDQN (1) ($\rho = -0.47$, $p < 0.001$, $n = 15$ per level). Points are means with standard-error bars.

That the harm is caused by the decoupling mechanism, rather than merely co-occurring with it, is established by intervening on the mechanism’s strength rather than toggling it. A continuous dial `dampened_double_greedify(alpha)` interpolates linearly between the two bootstrap values, from vanilla ($\alpha = 0$) to full DDQN ($\alpha = 1$). On Asterix at $\gamma = 0.999$, outcome falls monotonically as α rises, from 10.41 to 8.70 across five levels (Figure 2, Spearman $\rho = -0.47$, $p < 0.001$, $n = 15$ per level). The causal reading rests on the graded design, the monotone response to a continuously varied strength, rather than on the effect’s magnitude, which is modest.

We localise, but do not fully explain, why the mechanism is harmful here. The natural first move is observational. One conditions the DDQN-outcome relationship on a candidate mediator and asks which one absorbs it. Conditioning on the overestimation bias itself, the reading the literature would reach for, appears to succeed, absorbing most of the correlation at Asterix. But the Jensen gap is measured against the realised return, and the outcome aggregates the same realised return, so the absorption is partly an artefact of shared construction rather than evidence about the mechanism. No candidate channel that is independent of the outcome accounts for the harm, which is why we intervene rather than read the mechanism off an association.

Revisiting Double-DQN’s construction supplies the intervention. The target network it uses as its second estimator is a time-delayed copy of the online network rather than an independently trained one, so its decoupling of selection from evaluation is incomplete, a gap noted in recent work on adapting Double Q-learning to deep RL (Nagarajan et al., 2025). We use this as a single-edge probe. Holding the selector and the acting policy intact, we swap only the evaluator, from the time-delayed target to an independently trained network (DDQN-indp). The swap drives the residual overestimation at Asterix $\gamma = 0.999$ from a large positive value through zero to slightly negative (signed bias $+126.9$ to -4.9 , $n = 30$, Figure 3a), and it removes the harm, with return recovering to the vanilla baseline (discounted 15.7 against vanilla 12.4, Figure 3b). The bias change is categorical, from over-estimation to slight under-estimation, and it travels with the disappearance of the harm. We therefore implicate the target-side coupling, locally and causally, in the harm at this single environment. We make no claim that an independent evaluator is a generally better design, no claim about other environments, and no account of why the long horizon is the trigger. The localisation itself is the result. The observational analysis could not identify the harm’s mechanism, and the single-edge intervention did.

5 Conclusion

We have presented `corroborate`, a framework for the interventional study of algorithmic mechanisms as executable programs. A mechanism is isolated as a unit of intervention, a hypothesis is

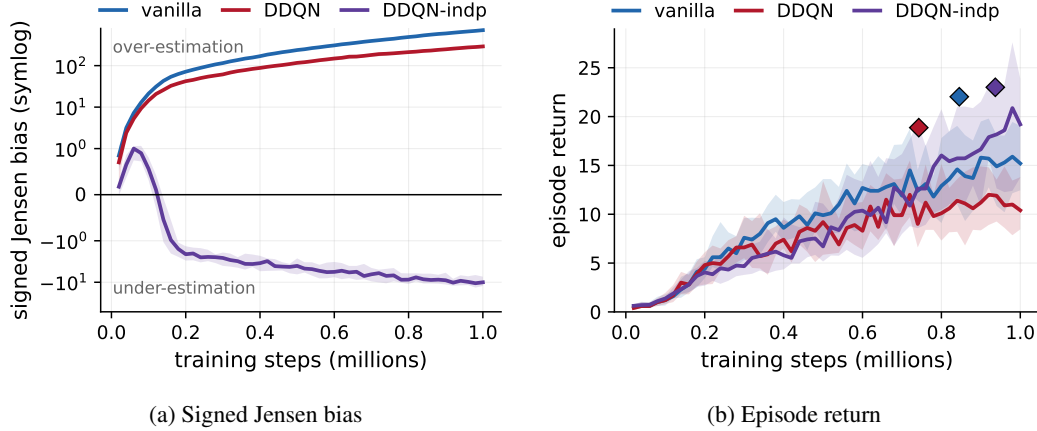


Figure 3: The single-edge evaluator swap on Asterix ($\gamma = 0.999$). Lines are medians over 30 seeds per condition and bands are interquartile ranges. (a) Unclipped signed Jensen bias on a symlog scale. An independently trained evaluator (DDQN-indp) drives the residual overestimation through zero, where vanilla’s bias grows large and DDQN only halves it. (b) Per-window mean episode return, with diamonds marking the mean per-seed peak. The harm disappears with the residual overestimation, as DDQN-indp’s return recovers to the vanilla baseline that DDQN sits below.

codified as an executable claim, and an inconclusive result propagates through composed claims instead of collapsing into "no effect."

We demonstrated the framework on Double-DQN, whose overestimation-bias mechanism is well understood. Most of what it reported is the field’s existing understanding, stated as explicitly scoped claims rather than a narrative. At $\gamma = 0.99$ the mechanism reduces bias across most environments, while its benefit to outcome is weak and environment-conditional. In one long-horizon regime the framework distinguished a working mechanism from a worse outcome. At Asterix $\gamma = 0.999$ the mechanism still reduced bias, yet the outcome fell. A single-edge intervention that swapped only the evaluator removed both the residual overestimation and the harm, locally implicating the target-side coupling as the cause.

The work has clear limits. The harmful regime is a single environment, and we draw no general law from it. The mediation analyses are underpowered in most environments, and the framework reports them as `POWER_INSUFFICIENT` rather than concealing the fact. The framework has also been exercised on only one algorithm family. Two further limits bear on adoption. First, the decomposition assumes that component boundaries can be drawn at theorems. Such boundaries survive in the RL fine-tuning of large pretrained models, where the objective (Rafailov et al., 2023), the advantage estimate (Shao et al., 2024), and the divergence penalty (Ouyang et al., 2022) are authored design decisions like the greedification slot. Scale instead strains the framework through power: a single-slot contrast at the seed counts of Appendix C needs tens of fine-tuning runs per condition, infeasible at that cost, and a study run there rests on the distinction of Section 3.3 between an absent effect and insufficient evidence, judged against thresholds fixed before the data were seen. The assumption fails only for mechanisms that are learned rather than authored, such as in-context learning, where no authored surface exists and a boundary must be discovered before it can be intervened on. That discovery is the subject of mechanistic interpretability (Olah et al., 2020), which this framework does not attempt; the isolation step of Section 3.1 does not transfer there, though the claim and verdict forms do not depend on how a boundary is found. Second, using `corroborate` on an existing algorithm requires authoring it into slot form, a structured re-implementation rather than a wrapper around existing code. The cost is bounded, since one decomposition per algorithm family is reusable and ours is released, but it is real. Applying the framework to a mechanism whose behaviour is genuinely unsettled is the next step.

References

- Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, and Marc G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- Marcin Andrychowicz, Anton Raichuk, Piotr Stańczyk, Manu Orsini, Sertan Girgin, Raphaël Marinier, Léonard Hussenot, Matthieu Geist, Olivier Pietquin, Marcin Michalski, Sylvain Gelly, and Olivier Bachem. What matters for on-policy deep actor-critic methods? a large-scale study. In *International Conference on Learning Representations (ICLR)*, 2021.
- Leemon Baird. Residual algorithms: Reinforcement learning with function approximation. In *International Conference on Machine Learning (ICML)*, pp. 30–37, 1995.
- Clément Bonnet, Daniel Luo, Donal Byrne, Shikha Surana, Sasha Abramowitz, Paul Duckworth, Vincent Coyette, Laurence I. Midgley, Elshadai Tegegn, Tristan Kalloniatis, Omayma Mahjoub, Matthew Macfarlane, Andries P. Smit, Nathan Grinsztajn, Raphael Boige, Cemlyn N. Waters, Mohamed A. Mimouni, Ulrich A. Mbou Sob, Ruan de Kock, Siddarth Singh, Daniel Furelos-Blanco, Victor Le, Arnau Pretorius, and Alexandre Laterre. Jumanji: a diverse suite of scalable reinforcement learning environments in JAX. In *International Conference on Learning Representations (ICLR)*, 2024.
- Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. Openai gym, 2016. arXiv:1606.01540.
- Rebecca DerSimonian and Nan Laird. Meta-analysis in clinical trials. *Controlled Clinical Trials*, 7(3):177–188, 1986.
- Logan Engstrom, Andrew Ilyas, Shibani Santurkar, Dimitris Tsipras, Firdaus Janoos, Larry Rudolph, and Aleksander Madry. Implementation matters in deep policy gradients: A case study on PPO and TRPO. In *International Conference on Learning Representations (ICLR)*, 2020.
- Julian P. T. Higgins and Simon G. Thompson. Quantifying heterogeneity in a meta-analysis. *Statistics in Medicine*, 21(11):1539–1558, 2002.
- Matthew W. Hoffman, Bobak Shahriari, John Aslanides, Gabriel Barth-Maron, Nikola Momchev, Danila Sinopalnikov, Piotr Stańczyk, Sabela Ramos, Anton Raichuk, Damien Vincent, Léonard Hussenot, Robert Dadashi, Gabriel Dulac-Arnold, Manu Orsini, Alexis Jacq, Johan Ferret, Nino Vieillard, Seyed Kamyar Seyed Ghasemipour, Sertan Girgin, Olivier Pietquin, Feryal Behbahani, Tamara Norman, Abbas Abdolmaleki, Albin Cassirer, Fan Yang, Kate Baumli, Sarah Henderson, Abe Friesen, Ruba Haroun, Alex Novikov, Sergio Gómez Colmenarejo, Serkan Cabi, Caglar Gulcehre, Tom Le Paine, Srivatsan Srinivasan, Andrew Cowie, Ziyu Wang, Bilal Piot, and Nando de Freitas. Acme: A research framework for distributed reinforcement learning, 2020. arXiv:2006.00979. <https://github.com/google-deeppmind/acme>.
- Shengyi Huang, Rousslan Fernand Julien Dossa, Antonin Raffin, Anssi Kanervisto, and Weixun Wang. The 37 implementation details of proximal policy optimization, 2022a. ICLR Blog Track.
- Shengyi Huang, Rousslan Fernand Julien Dossa, Chang Ye, Jeff Braga, Dipam Chakraborty, Kinal Mehta, and João G.M. Araújo. CleanRL: High-quality single-file implementations of deep reinforcement learning algorithms. *Journal of Machine Learning Research*, 23(274):1–18, 2022b. <https://github.com/vwxyzjn/cleanrl>.
- Scott M. Jordan, Adam White, Bruno Castro da Silva, Martha White, and Philip S. Thomas. Position: Benchmarking is limited in reinforcement learning research. In *International Conference on Machine Learning (ICML)*, 2024.
- Daniël Lakens. Equivalence tests: A practical primer for t tests, correlations, and meta-analyses. *Social Psychological and Personality Science*, 8(4):355–362, 2017.

- Robert Tjarko Lange. `gymnax`: A JAX-based reinforcement learning environment library, 2022. <https://github.com/RobertTLange/gymnax>.
- Martin A. Lindquist. Functional causal mediation analysis with an application to brain connectivity. *Journal of the American Statistical Association*, 107(500):1297–1309, 2012.
- Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Belle-mare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.
- Prabhat Nagarajan, Martha White, and Marlos C. Machado. Deep double Q-learning. *arXiv preprint arXiv:2507.00275*, 2025.
- Chris Olah, Nick Cammarata, Ludwig Schubert, Gabriel Goh, Michael Petrov, and Shan Carter. Zoom in: An introduction to circuits. *Distill*, 2020. <https://distill.pub/2020/circuits/zoom-in>.
- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- Andrew Patterson, Samuel Neumann, Martha White, and Adam White. Empirical design in reinforcement learning. *Journal of Machine Learning Research*, 25(318):1–63, 2024.
- Judea Pearl. *Causality: Models, Reasoning, and Inference*. Cambridge University Press, 2nd edition, 2009.
- Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D. Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021. <https://github.com/DLR-RM/stable-baselines3>.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.
- Hado van Hasselt. Double Q-learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2010.
- Hado van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double Q-learning. In *AAAI Conference on Artificial Intelligence*, 2016.
- Tyler J. VanderWeele and Eric J. Tchetgen Tchetgen. Mediation analysis with time varying exposures and mediators. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 79(3):917–938, 2017.
- Kenny Young and Tian Tian. Minatar: An atari-inspired testbed for thorough and reproducible reinforcement learning experiments, 2019. [arXiv:1903.03176](https://arxiv.org/abs/1903.03176).

A A hypothesis as code

DDQN’s chain, authored as a composed claim. The testable edges on this suite are the *link* (under vanilla, higher observed bias predicts worse outcome) and the *mechanism* (do(DDQN) reduces bias, where do(·) is Pearl’s intervention operator (Pearl, 2009)). Each edge is a *bridge*, a decorator declaring the claim’s commitments (source, target, predicted direction, evidential tier, and the scope of environments and conditions it applies to) over a body that maps a statistic to a verdict. The premise edge predicts the observed bias from the *dormancy gap* $\max(0, \sigma_Q \sqrt{2 \log |\mathcal{A}|} - B)$, the shortfall of the observed bias B below the structural floor that the theorem of van Hasselt (2010) implies for $|\mathcal{A}|$ actions with Q-estimate noise σ_Q . A zero gap means the premise is active, since the observed bias is at or above the floor. In this suite no environment exhibits a non-zero dormancy gap, so the edge is degenerate and the framework reports it as untestable rather than inventing a verdict. The link edge also has a late-window sibling that replaces the peak-window outcome with the mean over the last 30% of evaluation windows. The listing is an illustrative excerpt, with the premise and sibling bodies elided, and the full module is in the released code.²

```
# Link: under vanilla, higher observed bias predicts worse outcome
@claim_bridge(source='jensen_gap', target='eval_best_burst_raw_mean',
              direction=INVERSE, tier=ASSOCIATIONAL,
              scope=CANONICAL_DORMANCY_SCOPE & VANILLA_ONLY,
              predicted_direction='a<lt_b')
def bias_predicts_worse_outcome__vanilla(ps: PartialSpearmanResult):
    rho, p = ps.rho_pooled, ps.p_value
    if isnan(rho) or isnan(p): return POWER_INSUFFICIENT
    if rho <= -0.3 and p <= 0.05: return HELD
    if rho >= 0.3: return NO_EFFECT, SIGN_FLIP
    if abs(rho) <= 0.1: return NO_EFFECT, NULL_EFFECT
    return POWER_INSUFFICIENT

# Mechanism (intervention): do(DDQN) reduces bias, consistently cross-env
@claim_bridge(source=INTERVENTION, target='jensen_gap',
              direction=INVERSE, tier=INTERVENTIONAL,
              scope=CANONICAL_DORMANCY_SCOPE & PREMISE_ACTIVE_PER_STRATUM,
              predicted_direction='a<lt_b')
def ddqn_reduces_bias__consistently_cross_env(
    c: CrossEnvConsistencyBinomialResult):
    if c.n_strata_total < 5 or isnan(c.p_value): return POWER_INSUFFICIENT
    if c.p_value <= 0.05: return HELD
    if c.p_value <= 0.15: return POWER_INSUFFICIENT
    # below: refute, classified by direction of the majority
    ...

BRIDGES = (hasselt_floor_predicts_observed_bias__vanilla, # premise
           bias_predicts_worse_outcome__vanilla,         # link
           bias_predicts_worse_outcome__vanilla__late30, # link, late window
           ddqn_reduces_bias__consistently_cross_env)    # mechanism
```

Tiers. The link is ASSOCIATIONAL, a within-vanilla partial-Spearman correlation, so a reader knows not to read causation into it. The mechanism edge is INTERVENTIONAL. It rests on the single-component do(DDQN) swap of Section 3.1, and it is tested as a *directional consistency* claim across environments, a binomial sign-test on the per-environment effect, rather than as a random-effects pool (DerSimonian & Laird, 1986; Higgins & Thompson, 2002). The code calls these per-environment groups *strata*, hence `n_strata_total` and `PREMISE_ACTIVE_PER_STRATUM` in the listing. RL environments are not exchangeable draws from a population (they differ in network class, Q -magnitude, reward sparsity). A pooled prediction interval therefore does not extrapolate, but it also hides the cross-environment directional claim under a `NO_EFFECT` verdict. The sign-test asks the conditional question directly.

²<https://github.com/rnilva/corroborate>

What the verdicts are, and what they are not. On the $\gamma=0.99$ suite the link holds ($\rho = -0.31$, $p = 2.7e-8$) and the mechanism holds as a cross-environment consistency claim (10/12 environments, sign-test $p = 0.019$), while the *pooled* magnitude of the same mechanism effect is NO_EFFECT ($I^2 = 0.97$, prediction interval crossing zero). The direction is consistent, no poolable population magnitude exists, and the framework reports both rather than collapsing one into the other. Separately, DDQN’s *total effect* on outcome, direct plus indirect, is a structurally distinct claim (Pearl, 2009), and it is environment-conditional. This is why the chain holding as a mediated path does not justify a claim that the algorithm helps.

B Mechanism decomposition

The decomposition of Section 3.1, shown for the slot the Double-DQN intervention acts on. A Greedification *Protocol* fixes the call signature. Two @claim functions satisfy it, max_greedify (vanilla) and double_greedify (DDQN), so the intervention is one keyword and a type checker rejects a slot-filler of the wrong shape. The bootstrap composition holds the two orthogonal axes (greedification, gradient rule) as separate slots, and the DDQN swap touches only the first.

```
class Greedification(Protocol):
    """Compute v(s') from Q. The DDQN-vs-vanilla axis lives here."""
    def __call__(self, *, online_params: Params, target_params: Params,
                 q_network: QFunction, next_obs: Array) -> Array: ...

@claim
def max_greedify(*, online_params, target_params, q_network, next_obs):
    # vanilla: v(s') = max_a Q_target(s', a); online_params unused
    return jnp.max(q_network(target_params, next_obs), axis=-1)

@claim
def double_greedify(*, online_params, target_params, q_network, next_obs):
    # DDQN: v(s') = Q_target(s', argmax_a Q_online(s', a))
    a = jnp.argmax(q_network(online_params, next_obs), axis=-1)
    return take(q_network(target_params, next_obs), a)

@claim
def bootstrap(*, online_params, target_params, q_network, next_obs,
              reward, done, truncated, gamma,
              greedification: Greedification = max_greedify,
              gradient_rule: GradientRule = semi_gradient):
    # Bellman target: reward + gamma * (1 - terminated) * gr(greedify(...))
    v_next = greedification(online_params=online_params,
                           target_params=target_params,
                           q_network=q_network, next_obs=next_obs)
    terminated = done * (1.0 - truncated)
    return gradient_rule(reward + gamma * (1.0 - terminated) * v_next)

# the DDQN intervention is one keyword on the bootstrap slot:
# partial(bootstrap, greedification=double_greedify)
```

Why the boundary falls here. max_greedify and double_greedify differ by exactly one theoretical commitment, namely whether action selection and value evaluation share the target network (van Hasselt, 2010; van Hasselt et al., 2016), and nothing else. They have the same signature, the same return shape, and the same caller. The semi-gradient choice (Mnih et al., 2015) is a *separate* slot (gradient_rule), so an intervention on the overestimation mechanism cannot silently also change the gradient-flow rule. Operations that carry no theorem (parameter allocation, replay bookkeeping, the ϵ -schedule) live outside these slots as mechanics, and so are held fixed across the contrast by construction rather than by manual care.

C Environments and Hyperparameters

Environments. The four Atari-style environments (Asterix, Breakout, Freeway, SpaceInvaders) are the MinAtar miniaturisations of [Young & Tian \(2019\)](#), not the full Arcade Learning Environment games, and we use their JAX implementations from gymnax ([Lange, 2022](#)). gymnax also provides the classic-control tasks (CartPole, Acrobot, MountainCar) and the FourRooms and MetaMaze gridworlds. Snake and PacMan are from Jumanji ([Bonnet et al., 2024](#)). LunarLander is our own pure-JAX reimplementation of the Box2D task from Gym ([Brockman et al., 2016](#)), included in the released code so that it composes with the framework’s vectorised seed rollouts.

Shared configuration. Within each environment, all conditions share one training configuration, and only the intervened slot differs, so any outcome gap is attributable to that slot rather than to tuning. Every run uses DQN with Adam at learning rate 10^{-4} (100-step warmup, no weight decay), uniform replay with batch size 32, an ε -greedy behaviour policy annealed linearly from 1.0 to 0.05, and a hard target-network copy at a fixed interval. Vector-observation environments use an MLP with two 64-unit hidden layers. Grid-observation environments use a small CNN, with one 16-channel 3×3 layer and a 128-unit dense layer for MinAtar, and 8- and 16-channel 3×3 layers with a 64-unit dense layer for Jumanji. Evaluation is greedy, in windows of a fixed number of episodes at a fixed interval. Table 3 gives the per-environment settings of the $\gamma = 0.99$ calibration suite. Every per-run hyperparameter is also a column of the run cache shipped with the released code.

Long-horizon runs. The $\gamma = 0.999$ suite keeps the same optimiser, replay, network, and exploration settings. Run lengths are 10^6 steps, except LunarLander (2×10^6) and FourRooms (5×10^4). Every $\gamma = 0.999$ run evaluates 5 episodes per window, at an interval recorded in the shipped run cache, and uses 30 seeds per condition, except PacMan with 20. The single-edge intervention and the graded-strength experiment at Asterix $\gamma = 0.999$ both use the canonical Asterix configuration: the single-edge intervention with 30 seeds per condition, and the graded-strength experiment with 15 seeds per α level.

Environment	Source	Seeds	Steps	Replay	Sync	Eval every	Episodes
Acrobot	gymnax	30	10^5	5×10^4	100	2×10^3	20
CartPole	gymnax	30	10^5	5×10^4	100	2×10^3	1
MountainCar	gymnax	30	10^6	5×10^4	100	2×10^4	20
FourRooms	gymnax	30	10^5	5×10^4	100	2×10^3	20
MetaMaze	gymnax	30	10^6	5×10^4	100	2×10^4	20
LunarLander	this work	30	10^6	5×10^4	1000	2×10^4	20
Asterix	MinAtar	30	10^6	10^5	1000	2×10^4	5
Breakout	MinAtar	30	10^6	10^5	1000	2×10^4	20
Freeway	MinAtar	30	10^6	10^5	1000	2×10^4	20
SpaceInvaders	MinAtar	30	10^6	10^5	1000	2×10^4	1
Snake	Jumanji	30	3×10^6	5×10^4	1000	2×10^4	5
PacMan	Jumanji	10	10^6	5×10^4	1000	10^5	3

Table 3: Per-environment configuration of the $\gamma = 0.99$ calibration suite. Seeds are per condition. Replay is the buffer capacity, Sync the target-network copy interval in steps, Eval every the training-step interval between evaluation windows, and Episodes the greedy episodes per window. PacMan anneals ε over 10^4 steps, and all other environments use 10^5 . The MinAtar environments are played through gymnax.

Table 4 gives the per-environment suite composition.

Environment	$\gamma=0.99$ suite (§4.1, mech/link)	$\gamma=0.999$ suite (Table 2)	Intervention ($\gamma=0.999$)
Asterix	✓	✓	✓
Breakout	✓	✓	
SpaceInvaders	✓	✓	
FourRooms	✓	✓	
Freeway	✓	✓	
MetaMaze	✓	✓	
LunarLander	✓	✓	
PacMan	✓	✓	
Snake	✓		
CartPole	✓		
Acrobot	✓		
MountainCar	✓		

Table 4: Suite composition. The $\gamma=0.99$ calibration (§4.1) runs the full 12-environment set, and the mechanism’s cross-environment consistency verdict is 10/12 ($p=0.019$). The $\gamma=0.999$ analyses use the 8 environments with usable data at the long horizon (Table 2). Of these, the four MinAtar/grid environments (Asterix, Breakout, FourRooms, SpaceInvaders) reach adequate power for the dissociation contrast and are the subset reported in §4.2. The single-edge intervention (vanilla / DDQN / DDQN-indp, ≈ 30 seeds per condition, 10^6 steps) is run at Asterix $\gamma=0.999$. Seeds per condition are 30, except PacMan (10 at $\gamma=0.99$, 20 at $\gamma=0.999$), as Table 3 details.